

Simulação de Epidemias – PROLOG

GRUPO 3

Tainara Lopes de Oliveira Terassaka - 13675501

Guilherme Borges de Pádua Barbosa - 15653045

Gabriel Hupfer Righi - 15508612



Problema abordado – tópico 13:

Simulação de Epidemias – PROLOG

Este trabalho propõe uma simulação da propagação de uma doença infecciosa em uma população linear, com base em regras simples e fixas.

A população é representada como uma linha de indivíduos (P1, P2, ..., Pn), onde cada pessoa pode estar em um dos estados:

- **Suscetível**
- **Infectado**
- **Isolado**

Regras da Simulação:

- **O usuário insere a quantidade de pessoas totais e infectadas na população. Predicado `quant_pessoas/0`.**
- **Logo após o usuário insere taxa de mortalidade e de infecção da doença em porcentagem, além da quantidade de dias que a infecção dura no máximo, ou seja, com quantos dias a taxa de cura se torna 100%. Predicado `ler_taxas/0`.**
- **O usuário também insere as medidas de prevenção iniciais, ou seja, a quantidade de pessoas que iniciarão a simulação estando em um estado de ‘Isolado’ e também quantas pessoas estarão vacinadas. Predicado `medidas_prev/0`.**
- **A cada novo dia, as pessoas infectadas transmitem a infecção às suas conexões, tentam se curar da doença, e se não se curarem estarão sujeitas à taxa de morte.**

Organização:

- **Criamos regras em Prolog para definir os estados dinâmicos de cada pessoa a cada dia.**
- **O sistema é iterativo e baseado em fatos lógicos.**
- **A implementação considera regras fixas e simula dia a dia a propagação da doença.**



Principais algoritmos implementados

- **main/0** → Inicia o código mostrando o menu de entrada com as opções para o usuário inserir.
- **criar_pessoa/1** → Para um número N de pessoas, altera dinamicamente o predicado ‘estado’ com nomes de novas pessoas com a formatação ‘pN’ e o estado ‘suscetível’.
- **definir_conexoes/2** → Para todas as pessoas criadas, uma conexão é definida entre duas pessoas adjacentes, e a última pessoa é conectada à primeira na forma de uma lista circular.
- **ler_taxas/0** → Pede ao usuário quais serão as taxas de chance de infecção, duração da infecção e taxa de morte.
- **medidas_prev/0** → Pede ao usuário quantidade de pessoas vacinadas e em isolamento no primeiro dia, porém quais pessoas farão parte desses grupos é aleatória (uso de *random_between*).
- **loop_dias/0** → Inicia o ciclo de dias da simulação (limitado de 1 a 999 para evitar loops grandes demais) chamando o predicado “simular”.
- **simular** → Executa a infecção das pessoas conectadas, tenta curar quem está infectado, calcula quais pessoas vão morrer, diminui o tempo de isolamento quem está isolado e aumenta o tempo de infecção para todos os infectados.

Principais algoritmos implementados

- **tentar_infec/1** → Procura todas as conexões de uma pessoa infectada, verifica que se a conexão está suscetível, cria um número aleatório entre $[0,1]$ e compara com a taxa de infecção para enfim executar o predicado 'infectar/1' na conexão
- **infectar/1** → Infecta a pessoa, apagando o estado original e declarando que está infectada, define também o tempo de infecção para zero e aumenta o número de infectados.
- **curar/1** → Tenta curar a pessoa com base no tempo de infecção e em uma probabilidade crescente, em proporção quadrática, de acordo com o passar dos dias. Após ser curado se torna suscetível novamente.
- **vacinar/1** → Para uma pessoa saudável que não tenha sido vacinada anteriormente, define o estado dela como 'vacinado', diminuindo as taxas de infecção e morte de tal pessoa.
- **rodada_isolamento/0** → Atualiza o estado dos isolados, retirando as pessoas de isolamento que já cumpriram os dias e diminuindo os dias dos restantes para cada um em 1.
- **calc_taxa_morte/2** → Calcula a taxa de morte diária de acordo com a equação da parábola $f(x) = (-4T/D^2)x^2 + (4T/D)x$, onde as raízes são: $[0,D]$ sendo que o ápice é a taxa de morte inserida pelo usuário, crescendo do zero, no início, e alcançando seu maior valor na metade da infecção e diminuindo novamente até se tornar zero nos dias finais da infecção.
- **morte/1** → Declara a morte da pessoa, retira todos os seus estados e diminui o número de infectados e da população.



Utilização de LLM

Fizemos uso do Chat GPT para:

- Principalmente descobrir funções úteis para a realização de ações como:
 - a. procurar os infectados na lista de população (*findall*),
 - b. iterar sobre todos os infectados (*forall*),
 - c. concatenar átomo para formatar os estados de cada pessoa (*atomic_list_concat*)
 - d. função para evitar erros em loop (*ignore*).
- Correção pontual de sintaxe;



Dificuldades encontradas, vantagens e desvantagens do PROLOG neste contexto de modelamento de simulações

Dificuldades principais:

- A sintaxe do Prolog e a organização recursiva de regras é por vezes não intuitiva se comparada com outras línguas.
- Modelar a simulação sem redundância de código, com a menor quantidade de predicados e chamadas recursivas.
- Manter estados anteriores acessíveis para cálculo de estados futuros.

Vantagens do Prolog:

- Como relações lógicas são a base do Prolog, torna mais fácil fazer um código estruturado em regras.
- Simplicidade em simular comportamento baseado em condições.

Desvantagens:

- Pouca familiaridade da equipe com a linguagem e pouco conhecimento da diversidade de funções que poderiam ser utilizadas para facilitar a construção do código.
- Por vezes demorada compreensão e interpretação de erros.
- Requer raciocínio diferente do desenvolvido com linguagens de paradigma Orientado a Objetos ou mesmo Imperativo.



Problema abordado – tópico 13:

Simulação de Epidemias – LISP

- Este projeto modela a propagação de uma doença infecciosa em uma população linear.
- A população é representada como um vetor de indivíduos ($P_0, P_1, P_2, \dots, P_{n-1}$), onde cada pessoa pode estar nos estados: suscetível, infectado, recuperado, morto, isolado ou vacinado.

Regras da Simulação:

- Você declara o tamanho da população para simular a epidemia.
- A cada dia, cada infectado tem uma chance de 30% de transmitir a infecção a seus vizinhos diretos, 5% de chance de se recuperar e 3% de chance de morrer.
- A infecção dura 12 dias. Após esse período, o indivíduo se recupera e pode se tornar suscetível ou se vacinar.
- infectados têm chance de isolamento.
- Recuperados e suscetíveis têm chance de se vacinar.

Organização:

- Definição de variáveis globais e parâmetros.
- Entrada e validação de dados iniciais.
- Modelagem das interações sociais.
- Implementação das regras de transição entre estados.
- Execução diária da simulação até acabar os infectados.



Problema abordado – tópico 13:

Simulação de Epidemias

Organização:

- O usuário define o tamanho da população.
- A cada novo dia, indivíduos infectados podem infectar seus contatos diretos com base em uma taxa de infecção.
- A infecção dura até 10 dias. Após esse período, o indivíduo pode se recuperar ou morrer.
- Pessoas recuperadas podem ser infectadas novamente (chance pequena).
- Há chance de vacinação e isolamento baseados em determinada aleatoriedade.

Principais algoritmos implementados

- **tamanho-populacao():** recebe input do usuário e valida ou retorna mensagem de erro até digitar algo aceitável ao algoritmo;
- **qtd-conexoes(n):** calcula o número médio de conexões baseado em $\log(n)$, evitando que tenha situação de zero conexão;
- **gerar-conexoes():** cria conexões sociais evitando duplicatas e auto-conexão, armazenando isso como listas em um vetor;
- **infecta():** infecta contatos de infectados com base na taxa de infecção definida pelo usuário e remove duplicatas;
- **recupera():** recupera infectados por tempo limite da doença ou chance de recuperação;
- **reinfecta():** simula a perda de imunidade em recuperados com chance de 5%, fazendo-os voltar ao estado infectado.
- **morte():** avalia a possibilidade de morte dos infectados após cinco dias de doença, com base em uma probabilidade.
- **metodos-prevencao():** aplica vacinação e isolamento com probabilidades específicas por estado; retorna a lista de vacinados e isolados.
- **remover-isolados ():** remove as conexões associadas aos indivíduos isolados.
- **simular():** integra todos os passos por dia, atualizando estados da população.
- **mostrar-resultados():** imprime estatísticas do dia;
- **tem-infectados():** percorre a população e retorna T se houver ao menos um indivíduo nos estados infectado ou isolado.
- **main():** inicia simulação e define o paciente zero, iterando os dias até o fim da epidemia.



Utilização de LLM

Fizemos uso do Chat GPT para:

- Correção pontual de sintaxe;
- Encontrar e aplicar as funções *multiple-value-list*, *remove-duplicates*, *first*, *second*, *length* e *make-array*;
- Corrigir o problema da falta de variabilidade na geração aleatória a cada execução do código.



Dificuldades encontradas, vantagens e desvantagens do LISP neste contexto de modelamento de simulações

Dificuldades principais:

- A sintaxe e estrutura do LISP, especialmente o uso de *loop*, *setf*, *let* e estruturas aninhadas, foi um ponto que foi preciso reforçar para terminarmos o código;
- O controle manual de vetores e listas exigiu bastante atenção para usarmos corretamente *aref*, *append*, *remove-duplicates* etc;
- O escopo das variáveis também foi um problema na depuração.

Vantagens do Lisp:

- Temos bom controle sobre estruturas de dados e sobre o fluxo de execução, sendo mais flexível do que prolog;
- Pareceu ideal para simulações que exigem uma atualização constante de estados;
- Permitiu uma modularidade melhor do código (mais fácil de compreender e mais organizado).

Desvantagens:

- Pouca familiaridade da equipe com a linguagem.
- Necessidade de cuidar manualmente de estados e mutações em vetores.
- Requer raciocínio diferente do desenvolvido com linguagens de paradigma Orientado a Objetos ou mesmo Imperativo.



Obrigado

Tainara Lopes de Oliveira Terassaka - 13675501

Guilherme Borges de Pádua Barbosa - 15653045

Gabriel Hupfer Righi - 15508612

Link Repositório - <https://github.com/Guibpb/TrabalhoLPP>

